

Compiladores 2026 — Fase 2

Frontend/Backend, Código de Tres Direcciones, Optimización y Reportes HTML

1. Mapa de la presentación

1. Resumen ejecutivo: que se entrega en esta fase 2.
2. Arquitectura: separación **Frontend** / **Backend**.
3. **PAUSA teórica**: tipos de código intermedio y cuál usamos.
4. Código de Tres Direcciones (TAC).
5. Pipeline PLY: `lex` + `yacc` produciendo el AST.
6. Generación del TAC a partir del AST.
7. Análisis semántico — `check_semantic`.
8. **PAUSA pedagógica**: visualización y reportes HTML.
9. Optimización del TAC.
10. Pruebas y QA contra la rúbrica.

2. Resumen ejecutivo

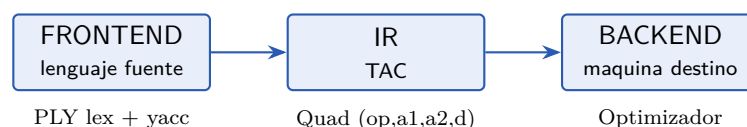
Que entregamos en la fase 2

- Reestructuración del código en `frontend/` y `backend/` con un contrato compartido (`intermedio.py`).
- **Generador de código de tres direcciones (TAC)** sobre el AST que produce el parser PLY `yacc`.
- **Análisis semántico** (`check_semantic`) que detecta tipos incompatibles, variables no declaradas, condiciones inválidas, división por cero, etc., con *línea y columna* por error.
- **Optimizador** con cinco pasadas iterando hasta punto fijo.
- **Reportes HTML** estructurados de tokens, tabla de símbolos, errores (con separado para semánticos), TAC original y TAC optimizado.
- UI dual: escritorio (`Tkinter`, `main.py`) y web (`index.php` + `analizar.php`) que muestran todo el pipeline al usuario.
- Suite de QA (`qa_rubrica.py`) con 42 casos automáticos cubriendo los cinco criterios de la rúbrica.

3. Arquitectura: Frontend / Backend

3.1. El principio

Un compilador moderno separa lo que depende del *lenguaje fuente* (frontend) de lo que depende de la *maquina destino* (backend). Entre los dos está la **representación intermedia** (IR), que es el contrato común.



3.2. Como lo aplicamos

compiladores-2026/

```

|-- frontend/                                <- depende del lenguaje fuente
| |-- __init__.py                            reexporta API publica
| |-- lexer.py                              PLY: tokens + tabla de simbolos
| |-- tabla_simbolos.py                     Simbolo + TablaSimbolos con ambitos
| |-- errores.py                           agregar_lexico/sintactico/semantico
| |-- parser.py                             PLY yacc: AST + check_semantic
| |-- generador_intermedio.py               AST -> TAC
|
|-- intermedio.py                           <- contrato compartido (IR)
| |-- Quad(op, arg1, arg2, dest), formatear_tac(...)
|
|-- backend/                                <- depende de la maquina objetivo
| |-- __init__.py
| |-- optimizador.py                        5 pasadas hasta punto fijo
|
|-- reportes.py                             reportes HTML (semanticos, TAC, etc.)
|-- bridge.py                              pipeline -> JSON
|-- main.py                                UI Tkinter
|-- index.php / analizar.php               UI web

```

```

# frontend/__init__.py
from .lexer import Lexer
from .tabla_simbolos import TablaSimbolos, Simbolo
from .parser import build_tree, get_kids, node_label, \
                    TIPOS_DATO, RESERVED, check_semantic
from .generador_intermedio import GeneradorTAC

# backend/__init__.py
from .optimizador import optimizar

```

Punto de discusion: intermedio.py no importa de frontend ni de backend. Es la *frontera*: cualquier modulo que respete Quad puede leer o escribir TAC.

4. PAUSA: tipos de codigo intermedio

Pausa teorica en la presentacion

Antes de mostrar nuestro generador de TAC conviene explicar **que tipos de codigo intermedio existen y cual elegimos**.

4.1. Los tres tipos clasicos

1. Notacion postfija (Reverse Polish)

La operacion se escribe *despues* de los operandos.

Ejemplo: $a + b * c$ se vuelve $a \ b \ c \ * \ +$.

Ventaja: compacta y facil de evaluar con una pila.

Desventaja: no representa flujo de control (saltos, etiquetas), solo expresiones aritmeticas.

2. Arbol Sintactico Abstracto (AST)

Estructura de arbol: cada nodo es una operacion o un literal y los hijos son los operandos.

Ejemplo: $a + b * c$ es un nodo $+$ cuyos hijos son a y un sub-nodo $*$ con hijos b y c .

Ventaja: mantiene la estructura del programa, ideal para analisis semantico.

Desventaja: no es lineal, no se ejecuta secuencialmente sin recorrerlo.

3. Código de Tres Direcciones (TAC)

Cada instrucción tiene a lo más tres operandos: dos fuentes y un destino. Forma canónica: `dest = arg1 op arg2`.

Ejemplo: `a + b * c` se descompone en dos cuádruplos.

Ventaja: lineal (una instrucción por línea), explícito (temporales con nombre), parecido al ensamblador.

Desventaja: más voluminoso que la postfija.

```
$t1 = b * c
$t2 = a + $t1
```

4.2. ¿Cuál usa nuestro compilador?

Decisión de diseño

El compilador produce y usa internamente **dos representaciones**:

1. **AST** producido por `build_tree()` en `frontend/parser.py`. Lo usamos para análisis semántico, para visualizar el árbol y como entrada del generador de TAC.
2. **Código de Tres Direcciones (TAC)** producido por `GeneradorTAC.generar()` en `frontend/generador_intermedio.py`. Es el formato sobre el que trabaja el optimizador (`backend/optimizador.py`).

No usamos notación postfija porque no permite expresar saltos ni asignaciones, que son básicos en este lenguaje.

5. Código de Tres Direcciones

5.1. Estructura del cuádruplo

```
# intermedio.py
@dataclass
class Quad:
    op: str                # operacion o palabra clave
    arg1: Optional[str] = None # primer operando
    arg2: Optional[str] = None # segundo operando
    dest: Optional[str] = None # destino o etiqueta destino
```

5.2. Conjunto de instrucciones del IR

Asignación:	<code>dest = arg1</code>	<code>op = "="</code>
Operación binaria:	<code>dest = arg1 op arg2</code> <code>{+, -, *, /, %, ==, !=, <, >, <=, >=, &&, }</code>	<code>op in</code>
Operación unaria:	<code>dest = ! arg1</code>	<code>op = "!"</code>
Etiqueta:	<code>L:</code>	<code>op = "label", dest = L</code>
Salto:	<code>goto L</code>	<code>op = "goto", dest = L</code>
Salto condicional:	<code>ifFalse arg1 goto L</code>	<code>op = "if_false", dest = L</code>
Imprimir:	<code>imprimir arg1</code>	<code>op = "print"</code>
Leer:	<code>leer dest</code>	<code>op = "read", dest = nombre</code>

5.3. Pretty-printer `formatear_tac`

```
def formatear_tac(quads):
    filas, n = [], 0
    for q in quads:
        op, a1, a2, d = q.op, q.arg1, q.arg2, q.dest
        if op == "label":
            instr = f"{d}:"
        elif op == "goto":
            instr = f"goto {d}"
```

```

elif op == "if_false": instr = f"ifFalse {a1} goto {d}"
elif op == "=":       instr = f"{d} = {a1}"
elif op == "print":   instr = f"imprimir {a1}"
elif op == "read":    instr = f"leer {d}"
elif op == "!":       instr = f"{d} = !{a1}"
else:                 instr = f"{d} = {a1} {op} {a2}"
n += 1
filas.append({"n": n, "instruccion": instr,
              "op": op, "arg1": a1 or "-",
              "arg2": a2 or "-", "dest": d or "-"})

return filas

```

6. Pipeline PLY: lex + yacc

6.1. Lexer con PLY

```

# frontend/lexer.py
import ply.lex as lex
from . import errores as _errores_mod

reservadas = {
    "programa": "PROGRAMA",
    "entero": "ENTERO", "decimal": "DECIMAL",
    "cadena": "CADENA_TIPO", "booleano": "BOOLEANO",
    "si": "SI", "sino": "SINO",
    "mientras": "MIENTRAS", "hacer_mientras": "HACER_MIENTRAS",
    "para": "PARA", "imprimir": "IMPRIMIR", "leer": "LEER",
    "verdadero": "VERDADERO", "falso": "FALSO",
    "funcion": "FUNCION", "procedimiento": "PROCEDIMIENTO",
    "retornar": "RETORNAR",
}

t_MAS = r'\+'; t_MENOS = r'\-'; t_MULTIPLICACION = r'\*';
t_DIVIDIR = r'\('; t_MODULO = r'%';
t_IGUAL = r'=='; t_DIFERENTE = r'!=';
t_MENOR_IGUAL = r'<='; t_MAYOR_IGUAL = r'>=';
t_AND = r'&&'; t_OR = r'\|\|'; t_NOT = r'!'
...

def t_IDENTIFICADOR(t):
    r'[a-zA-Z_][a-zA-Z0-9_]*'
    t.type = reservadas.get(t.value, 'IDENTIFICADOR')
    return t

def t_error(t):
    col = encontrar_columna(t.lexer.lexdata, t)
    _errores_mod.agregar_lexico(
        f"Caracter ilegal '{t.value[0]}'",
        t.lineno, col, valor=t.value[0],
    )
    t.lexer.skip(1)

```

Que hace:

- Define tokens con regex como atributos `t_XXX` (PLY los recoge por convencion de nombre).
- Las palabras reservadas se distinguen del identificador *post-match*: si el `IDENTIFICADOR` matchea "programa", se reclasifica a `PROGRAMA`.
- Los errores lexicos se acumulan en el modulo `errores` (con linea y columna).

6.2. Parser con PLY yacc

```
# frontend/parser.py
import ply.yacc as yacc

precedence = (
    ('left', 'OR'),
    ('left', 'AND'),
    ('left', 'IGUAL', 'DIFERENTE'),
    ('left', 'MENOR', 'MAYOR', 'MENOR_IGUAL', 'MAYOR_IGUAL'),
    ('left', 'MAS', 'MENOS'),
    ('left', 'MULTIPLICACION', 'DIVIDIR', 'MODULO'),
    ('right', 'NOT', 'UMENOS'),
)

def p_programa(p):
    '''programa : PROGRAMA LLAVE_IZQ sentencias LLAVE_DER
                | sentencias'''
    if len(p) == 5:
        p[0] = {"type": "Program", "children": p[3]}
    else:
        p[0] = {"type": "Program", "children": p[1]}

def p_declaracion(p):
    '''declaracion : tipo IDENTIFICADOR ASIGNAR expresion PUNTO_COMA'''
    p[0] = {
        "type": "Declaration",
        "dataType": p[1],
        "id": _tok_de_p(p, 2, tipo="IDENTIFICADOR"),
        "expr": p[4],
    }

def p_expression_binaria(p):
    '''expression : expression MAS expression
                  | expression MENOS expression
                  ...
                  | expression AND expression
                  | expression OR expression'''
    op_tok = {"tipo": p.slice[2].type,
              "valor": _OP_TIPOS.get(p.slice[2].type, str(p[2])),
              "linea": p.lineno(2), "columna": col}
    p[0] = {"type": "BinaryOp", "op": op_tok, "left": p[1], "right": p[3]}
```

Decision clave: las acciones semanticas del parser *no producen tuplas* (como hacia la fase 1) sino **dicts en formato AST** que el resto del pipeline (visitor del TAC, semantico, visualizacion del arbol) ya entiende.

6.3. Sintaxis del lenguaje

```
programa {
    entero edad = 25;
    decimal salario = 15750.50;
    cadena nombre = "Ana Garcia";
    booleano activo = verdadero;

    si (edad >= 18 && activo) {
        imprimir(nombre);
    } sino {
        imprimir("menor");
    }

    mientras (edad > 0) { edad = edad - 1; }
    para (entero i = 0; i < 5; i = i + 1) { imprimir(i); }
}
```

7. Generacion del TAC

7.1. Visitor por tipo de nodo

```
class GeneradorTAC:
    def __init__(self):
        self.codigo: list[Quad] = []
        self._t = 0      # contador de temporales
        self._l = 0      # contador de etiquetas

    def _nuevo_temp(self): self._t += 1; return f"${self._t}"
    def _nueva_etiq(self): self._l += 1; return f"$L{self._l}"
    def emit(self, op, a1=None, a2=None, d=None):
        self.codigo.append(Quad(op, a1, a2, d))

    def generar(self, ast):
        self.codigo = []; self._t = 0; self._l = 0
        if ast: self._gen(ast)
        return self.codigo

    def _gen(self, node):
        if not node: return None
        method = getattr(self, f"_g_{node['type']}", self._g_default)
        return method(node)
```

Patron visitor: para un nodo If se llama `_g_If`, para BinaryOp se llama `_g_BinaryOp`, y asi. Sin if/elif repetitivos.

7.2. Expresiones binarias en post-orden

```
if t == "BinaryOp":
    izq = self._g_expr(node["left"])
    der = self._g_expr(node["right"])
    tmp = self._nuevo_temp()
    op = _BIN_TIPOS.get(node["op"]["tipo"], node["op"]["valor"])
    self.emit(op, izq, der, tmp)
    return tmp
```

Como funciona: primero se genera el TAC del subarbol izquierdo y se obtiene el lugar donde quedo su resultado; lo mismo con el derecho; despues se emite `tmp = izq op der`. Los temporales se encadenan naturalmente.

7.3. Estructuras de control

Si / sino:

```
def _g_If(self, node):
    cond = self._g_expr(node["cond"])
    L_else = self._nueva_etiq()
    L_end = self._nueva_etiq()
    self.emit("if_false", cond, None, L_else)
    self._gen(node["body"])
    if node.get("elseBody"):
        self.emit("goto", None, None, L_end)
        self.emit("label", None, None, L_else)
        self._gen(node["elseBody"])
        self.emit("label", None, None, L_end)
    else:
        self.emit("label", None, None, L_end)
```

Mientras:

```
def _g_While(self, node):
    L_inicio = self._nueva_etiq()
    L_fin     = self._nueva_etiq()
    self.emit("label",      None, None, L_inicio)
    cond = self._g_expr(node["cond"])
    self.emit("if_false",   cond, None, L_fin)
    self._gen(node["body"])
    self.emit("goto",       None, None, L_inicio)
    self.emit("label",     None, None, L_fin)
```

7.4. Ejemplo end-to-end

Fuente:

```
programa {
    entero a = 3;
    entero b = 4;
    entero c = (a + b) * 2;
    si (c > 10) { imprimir(c); } sino { imprimir("pequeno"); }
}
```

TAC generado (12 cuádruplos):

```
1: a = 3
2: b = 4
3: $t1 = a + b
4: $t2 = $t1 * 2
5: c = $t2
6: $t3 = c > 10
7: ifFalse $t3 goto $L1
8: imprimir c
9: goto $L2
10: $L1:
11: imprimir "pequeno"
12: $L2:
```

TAC optimizado (7 cuádruplos — reduccion 41.7%):

```
1: a = 3
2: b = 4
3: c = 14
4: imprimir 14
5: goto $L2
6: imprimir "pequeno"
7: $L2:
```

Punto de discusion: la propagacion de constantes y el folding calculan $(3+4)*2 = 14$ en tiempo de compilacion. Los temporales muertos se eliminan. La *rama muerta* (linea 6) se conserva porque no podemos garantizar staticamente que ese imprimir no sera alcanzado desde otro flujo, pero los temporales asociados ya estan plegados.

8. Analisis semantico — check_semantic

Criterio 1 de la rubrica (25 pts)

La rubrica exige detectar errores semanticos. Nuestro `check_semantic` recorre el AST e identifica:

- Variables **no declaradas** en uso, asignacion o lectura.
 - Variables **duplicadas** en el mismo ambito (lo detecta la tabla de simbolos al insertar).
 - **Tipos incompatibles** en declaracion o asignacion (`entero = "hola"`, `booleano = 5...`).
 - **Condiciones** de `si/mientras/ hacer_mientras/para` que no son booleanas.
 - **Division por cero** con denominador literal.
 - **Operacion aritmetica** entre tipos incompatibles (e.g. `booleano + entero`).
 - **Comparaciones y operaciones logicas** entre tipos invalidos.
 - **Operadores unarios** - y ! aplicados a tipo erroneo.
 - Funcion no definida en llamadas.
 - *Widening* permitido: `decimal = entero` se acepta sin error (promocion `entero → decimal`).
- Cada error reporta linea y columna.

8.1. Compatibilidad de tipos

```
def _tipo_compatible(declarado, expresion):
    if declarado == expresion:
        return True
    if declarado == "decimal" and expresion == "entero":
        return True # widening entero -> decimal
    return False
```

8.2. Inferencia de tipos en expresiones

```
def get_type(node):
    if t == "Literal": return _tipo_literal(node["token"])
    if t == "StringLit": return "cadena"
    if t == "BoolLit": return "booleano"
    if t == "Identifier":
        sym = tabla.buscar(node["token"]["valor"])
        if not sym:
            _sem("Variable '...' no declarada", linea, col)
            return None
        return sym.tipo
    if t == "BinaryOp":
        lt = get_type(node["left"]); rt = get_type(node["right"])
        op = node["op"]["tipo"]
        # Division por cero
        if op in ("DIVIDIR", "MODULO") and node["right"].get("type") == "Literal":
            if float(node["right"]["token"]["valor"]) == 0:
                _sem("Division por cero", op_lin, op_col)
        # Aritmetica entre numericos
        if op in ("MAS", "MENOS", "MULTIPLICACION", "DIVIDIR", "MODULO"):
            if lt == "entero" and rt == "entero": return "entero"
            if lt in ("entero", "decimal") and rt in ("entero", "decimal"):
                return "decimal"
            _sem(f"Operacion aritmetica incompatible entre '{lt}' y '{rt}'",
                op_lin, op_col)
            return None
    ...
```

8.3. Reglas por sentencia

```
def check_node(node):
    if t == "Declaration":
        et = get_type(node["expr"])
        vt = _TIPO_VAR_DE_TOKEN[node["dataType"]]["tipo"]
        if et and not _tipo_compatible(vt, et):
            _sem(f"Asignacion incompatible: '{vt}' no puede recibir '{et}'",
                linea, col)
    elif t == "Assignment":
        sym = tabla.buscar(node["id"]["valor"])
        if not sym:
            _sem(f"Variable no declarada", linea, col)
        else:
            et = get_type(node["expr"])
            if et and not _tipo_compatible(sym.tipo, et):
                _sem(f"Asignacion incompatible", linea, col)
    elif t == "If":
        ct = get_type(node["cond"])
        if ct and ct != "booleano":
            _sem(f"Condicion del 'si' debe ser booleana, no '{ct}'", ...)
        check_node(node["body"])
    ...
```

9. PAUSA: visualizacion y reportes HTML

Pausa pedagogica

Aqui se hace pausa para mostrar como el compilador *exhibe* su trabajo. Esto cumple dos criterios de la rubrica: el reporte HTML de errores semanticos (criterio 2, 20 pts) y el funcionamiento general del compilador (criterio 5, 10 pts).

9.1. UI Tkinter (main.py)

Una sola clase MiniIDE(tk.Tk) con seis pestanias:

- **Editor** con coloreado de tokens en vivo.
- **Tokens** con tipo, valor, linea y columna.
- **Arbol** (AST) renderizado en canvas.
- **Semantico** con tabla de simbolos.
- **Codigo Intermedio** con TAC original, TAC optimizado y metricas.
- **Errores** con consola dedicada.

Atajo: F5 = Analizar. Boton **REPORTES HTML** en la barra inferior que invoca `reportes.generar_reportes_`

9.2. UI web (index.php + analizar.php)

```
# analizar.php llama a bridge.py via exec()
$tmpFile = tempnam(sys_get_temp_dir(), 'minicomp_') . '.programa';
file_put_contents($tmpFile, $codigo);
$python = (PHP_OS_FAMILY === 'Windows') ? 'python' : 'python3';
$cmd = escapeshellcmd($python).' '
      . escapeshellarg(__DIR__ . '/bridge.py').' '
      . escapeshellarg($tmpFile).' 2>&1';
exec($cmd, $output, $returnCode);
@unlink($tmpFile);
echo json_encode(json_decode(implode("\n", $output), true), JSON_UNESCAPED_UNICODE
);
```

9.3. reportes.py: HTML estructurado

Criterio 2 de la rubrica (20 pts) — el HTML semantico

El archivo `reportes.py` genera HTMLs profesionales con CSS moderno y consistente. La rubrica pide especificamente un reporte *estructurado, claro y funcional con linea y columna de cada error semantico*: ese es `generar_html_errores_semanticos()`.

Funciones expuestas:

```
# reportes.py
generar_html_errores_semanticos(errores, ruta)    # << el clave (rubrica)
generar_html_errores(errores, ruta)              # todos los errores
generar_html_tokens(tokens, ruta)
generar_html_tabla_simbolos(tabla, ruta)
generar_html_tac(quads_formateados, ruta, titulo)
generar_reportes_completos(directorio, ...)      # todo de una
```

Lo que muestra el reporte semantico:

- Encabezado con fecha y resumen estadistico por categoria (variables no declaradas, tipos incompatibles, division por cero, condiciones no booleanas, etc.).
- Tabla principal con columnas: #, tipo (tag coloreado), **linea**, **columna**, identificador afectado, descripcion.
- Mensaje verde de "sin errores" si la lista esta vacia.

Estructura del HTML (extracto del CSS):

```
header { gradient azul + titulo + fecha }
.summary { chips estadisticos por categoria }
table {
  thead   th   bg=primario, fg=blanco
  tbody  alternancia de filas, hover, monospace para identificadores
  .tag    coloreado por tipo (lex=rojo, sint=amarillo, sem=morado)
}
```

Como se genera:

- Desde Tkinter: boton "REPORTES HTML" → abre el HTML de errores semanticos en el navegador.
- Desde CLI: `python bridge.py archivo.programa -reportes salida/` crea todos los archivos en `salida/`.

10. Optimizacion del TAC

10.1. Driver con punto fijo

```
def optimizar(codigo, max_iter: int = 12):
    actual = deepcopy(codigo); traza = []
    pasadas = [
        ("Constant Folding & Algebraic", _constant_folding),
        ("Constant / Copy Propagation", _propagation),
        ("Branch Pruning", _branch_pruning),
        ("Dead-Code Elimination", _dead_code),
        ("Jump Threading", _jump_threading),
    ]
    for it in range(max_iter):
        cambio_global = False
        for nombre, fn in pasadas:
            antes = len(actual)
            actual, cambio = fn(actual)
```

```

        if cambio:
            traza.append({"iter": it+1, "pasada": nombre,
                        "antes": antes, "despues": len(actual),
                        "delta": len(actual) - antes})
            cambio_global = True
        if not cambio_global: break
    return actual, traza

```

Que hace: aplica las cinco pasadas *en orden* y repite el ciclo hasta que una iteracion completa no haga ningun cambio (punto fijo). Cada modificacion se registra en `traza`.

10.2. Pasada 1 — Constant Folding

Idea: si los dos operandos son constantes, evaluar en tiempo de compilacion. Si uno es identidad algebraica ($x+0$, $x*1$, $x*0$), reescribir.

```

if q.op == "+" and a1 == "0":
    nuevo.append(Quad("=", a2, None, q.dest)); cambio=True; continue
if q.op == "*" and (a1 == "0" or a2 == "0"):
    nuevo.append(Quad("=", "0", None, q.dest)); cambio=True; continue
if _es_numero(a1) and _es_numero(a2):
    r = _BIN_OPS[q.op](_to_num(a1), _to_num(a2))
    nuevo.append(Quad("=", _fmt(r), None, q.dest)); cambio=True

```

10.3. Pasada 2 — Constant / Copy Propagation

Idea: mantener un entorno con los valores conocidos. Cuando se ve $x = 5$, recordar `env["x"] = "5"` y reemplazar usos posteriores. El entorno se invalida al cruzar etiquetas o saltos.

10.4. Pasada 3 — Branch Pruning

Idea: si la propagacion dejo la condicion de un salto como literal (verdadero/falso), eliminar el salto inutil o convertirlo en goto incondicional.

```

_VERDADERO = frozenset(("true", "True", "1", "verdadero", "Verdadero"))
_FALSO     = frozenset(("false", "False", "0", "falso", "Falso"))

if q.op == "if_false":
    if q.arg1 in _VERDADERO: cambio=True; continue # nunca salta
    if q.arg1 in _FALSO:     cambio=True; continue # siempre salta
    nuevo.append(Quad("goto", None, None, q.dest)); cambio=True

```

10.5. Pasada 4 — Dead-Code Elimination

Idea: eliminar los cuádruplos cuyo destino es un temporal que no se lee en ningun lado.

```

leidos = {a for q in codigo for a in (q.arg1, q.arg2) if a}
for q in codigo:
    if q.op == "=" or q.op in _BIN_OPS or q.op == "!":
        if q.dest and _es_temp(q.dest) and q.dest not in leidos:
            cambio=True; continue # se descarta
    nuevo.append(q)

```

Importante: solo se eliminan temporales ($\$tN$). Las variables del usuario nunca se borran aunque parezcan muertas.

10.6. Pasada 5 — Jump Threading

Idea (a): si encontramos `goto L` seguido de `L:`, eliminamos el `goto`.

Idea (b): las etiquetas que ya no son referenciadas se borran.

10.7. Como cooperan las pasadas

El truco del punto fijo

1. Folding pliega $\$t1 = 3 + 4$ a $\$t1 = 7$.
 2. Propagation reemplaza $\$t2 = \$t1 * 2$ por $\$t2 = 7 * 2$.
 3. Folding (siguiente iteracion) pliega a $\$t2 = 14$.
 4. Propagation reemplaza $c = \$t2$ por $c = 14$.
 5. Dead-code elimina $\$t1$ y $\$t2$ muertos.
 6. Jump threading limpia etiquetas huérfanas si quedan.
- Por eso iteramos hasta punto fijo en lugar de una sola pasada.

11. Pruebas y QA: qa_rubrica.py

Cobertura contra la rubrica

Suite de **42 casos automaticos** cubriendo los cinco criterios tecnicos. Resultado actual: **42/42 PASS (100 %)**.

Desglose:

Criterio 1. Errores semanticos	20/20 casos PASS	(100 %)
Criterio 2. Reporte HTML	2/2 casos PASS	(100 %)
Criterio 3. TAC	13/13 casos PASS	(100 %)
Criterio 4. Optimizacion	5/5 casos PASS	(100 %)
Criterio 5. Funcionamiento	2/2 casos PASS	(100 %)

	42/42 PASS	

Ejemplos de los casos cubiertos (criterio 1):

- Variable no declarada (uso simple y en asignacion).
- Variable duplicada en mismo ambito.
- Asignaciones `int=cadena`, `int=decimal`, `bool=int`.
- Widening `decimal=entero` permitido.
- Condicion de `si/mientras/para` no booleana.
- Division y modulo por cero literal.
- Suma entre booleano y entero.
- Comparacion entre tipos incompatibles.
- Operacion logica entre no booleanos.
- Operadores `-` y `!` con tipos invalidos.

12. Cierre: cosas para senalar al presentar

1. Mostrar el arbol de carpetas. Enfatizar que `intermedio.py` es la frontera.
2. Hacer la **pausa teorica** sobre los tres tipos de IR. Escribir el ejemplo $(a+b)*2$ en pizarra.
3. Mostrar TAC original y TAC optimizado lado a lado. Enfatizar *cuantos cuádruplos se redujeron y que pasada lo logro*.
4. Demostrar la traza del optimizador (iter, pasada, antes, despues, delta).
5. Cargar un programa con errores semanticos en la UI y mostrar que *el TAC no se genera* hasta arreglarlos.

6. Hacer click en "REPORTES HTML" y abrir el reporte de errores semanticos en el navegador.
Resaltar linea y columna.
7. Mostrar la corrida de `python qa_rubrica.py` con los 42/42 PASS.
8. Cerrar con la idea de **punto fijo**: las pasadas se alimentan entre si, por eso iteramos.